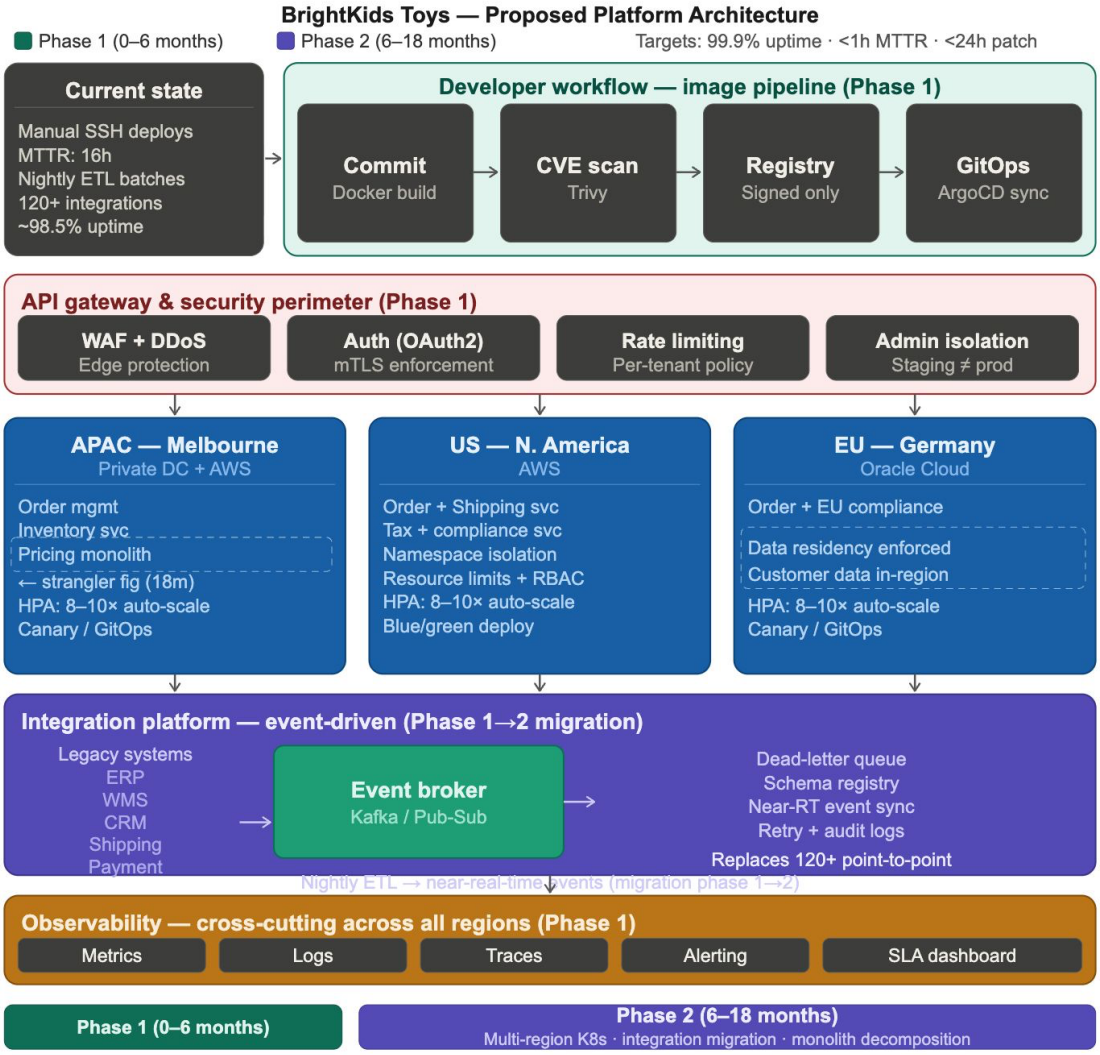
BrightKids Toys — Platform Modernisation Plan

Name: Bharath Arun Gandhimani **Student No:** 35501308

Architecture Diagram



Q1. Phased Architecture Roadmap

Phase 1: 0 to 6 months: Stabilise and Secure

Looking at the three incidents from last holiday season — the 16-hour US order delay, European pricing errors from config drift, and the exposed admin endpoint in staging — they are not three separate problems. They all trace back to the same underlying gap: there is no standardised way to build, deploy, or manage services across BrightKids' regions. Phase 1 does not attempt a full architectural overhaul. It fixes the specific infrastructure gaps that made those incidents inevitable.

The starting point has to be the container image pipeline, before anything else is migrated. The 11-day CVE patching cycle is the clearest evidence of what happens without one. Teams had different base images, no shared registry, and no clear ownership for emergency rollouts. The proposed pipeline requires every service build to start from a centrally approved base image, pass through automated vulnerability scanning via Trivy, land in a signed-only registry, and deploy exclusively through ArgoCD. This structurally eliminates manual SSH deployments, not as a rule engineers are asked to follow, but as a constraint the system enforces. A critical CVE then becomes a single base image update that cascades into automated rebuilds globally, which is how you get from 11 days to under 24 hours.

Containerisation starts with the stateless services: order capture and inventory/shipping estimation. They handle the most traffic, have the clearest deployment boundaries, and carry no state dependencies on the pricing monolith. The monolith itself does not move in Phase 1. Attempting to decompose it while simultaneously maintaining 99.9% uptime on order APIs in the six months before the next holiday season is a risk that is not worth taking. The sequencing matters here.

Kubernetes is brought in for one region only, APAC/Melbourne, but with full governance in place from the start. The pilot cluster BrightKids already has is essentially ungoverned: no standardised naming, no resource limits, no readiness probes. That is not a starting point, it is a problem to fix. The new setup uses standardised cluster templates with namespace isolation, RBAC, resource limits, and HPA tuned for 8-10x traffic spikes. All deployments go through ArgoCD, which means no engineer is directly touching production clusters. This is the practical answer to leadership's concern about Kubernetes complexity. The complexity is absorbed by automation, not pushed onto a limited engineering team.

The API gateway goes in as a hard security perimeter across all traffic. The pen test finding, an admin endpoint reachable from a misconfigured network path, was not just a misconfiguration. It was a symptom of having no enforced boundary between environments. The gateway enforces OAuth2 on all inbound requests, mTLS between internal services, and network policies that make staging and production structurally separate. The admin endpoint exposure cannot recur because the network path that allowed it no longer exists.

Observability is Phase 1, not Phase 2. The US inventory sync failure went undetected for hours because there was no central monitoring across regions. Metrics, logs, and distributed traces aggregated into a single dashboard with automated alerting on integration failures means that failure gets caught in minutes, not after customers start complaining.

Phase 2: 6 to 18 months: Scale and Migrate

With the foundation stable, Phase 2 extends the Kubernetes footprint to North America on AWS and Germany on Oracle Cloud, using the same cluster templates established in Melbourne. HPA configurations include pre-scaling policies timed to known peak periods. The 8-10x spikes BrightKids sees during product launches happen fast enough that reactive scaling is not a reliable response. Blue/green and canary deployments become the default release pattern across all regions, which is what makes a shift from manual infrequent releases to weekly or on-demand deployments actually safe rather than just aspirational.

The pricing and promotions monolith gets decomposed using a strangler fig approach. New services, including regional pricing rules, discount engine, and promotional campaign logic, are built in front of the monolith and intercept specific request types. The monolith keeps running and handling everything else. Over time its scope shrinks until what is left is small enough to retire cleanly, with no big-bang cutover. This is the only viable approach given that the monolith is business-critical and BrightKids cannot pause order operations.

The nightly ETL warehouse sync is replaced by near-real-time event streaming through the integration platform. The ETL job runs in parallel during the transition while event-driven output is validated against it. The batch process is not switched off until parity in production is confirmed. For the EU cluster, data residency under GDPR is enforced through cluster-level network policies that prevent customer data from leaving Oracle Cloud Frankfurt. This is embedded in cluster configuration, not dependent on engineers remembering to do the right thing.

Q2. API Governance Framework

The penetration test result was described as a security finding, but the real issue was a governance one. There was no defined ownership of the boundary between staging and production, which is why an admin endpoint ended up reachable from a misconfigured network path in the first place. Fixing authentication on that one endpoint is not the solution. The solution is making that class of mistake structurally impossible.

Every API, internal or external, is registered through the centralised gateway. Internal APIs are not accessible from external network paths unless explicitly allowlisted. Admin endpoints sit behind an additional isolation layer, network policy enforcement on top of authentication, not instead of it. Staging runs on a separate network segment with no routes to production data, so the specific scenario from the pen test cannot happen again regardless of configuration drift.

Authentication is handled at the gateway layer centrally, not reimplemented per service. OAuth2 for external and partner-facing APIs, mTLS for internal service-to-service communication. This matters because the current situation, where services implement their own auth checks inconsistently, is precisely how weak authentication ends up on an admin endpoint. Centralising it removes that variability. Rate limiting is applied per tenant and per endpoint, which also provides the first line of defence against traffic spikes that would otherwise hit clusters directly.

Base image governance is part of this framework. A central catalogue of approved base images is maintained by a designated platform team. The signed-only registry blocks deployments from unapproved images at the pipeline gate. This is not a review process, it is an automated enforcement point. Emergency patching carries an internal SLA: base image updated and validated within 4 hours of a critical CVE, with automated global rebuilds completing within 24 hours.

Partner-facing APIs are versioned with mandatory deprecation windows for breaking changes, and they sit in a separate tier at the gateway level with their own rate limit pools. This prevents a retail partner's traffic surge from affecting internal order management throughput, a separation that does not currently exist.

Q3. Integration Platform Recommendation

The recommended approach is a hybrid event-driven platform. Not a pure iPaaS subscription, and not a fully custom internal build.

The reason pure iPaaS does not work here is that BrightKids' 120+ integrations span legacy ERP systems, WMS, CRM, shipping vendors, payment providers, and regional compliance systems, many of which run on older protocols that an off-the-shelf iPaaS handles poorly. More importantly, a pure iPaaS creates deep vendor dependency on a connector catalogue that was not designed for the high-volume, low-latency event requirements of replacing a nightly ETL with real-time warehouse sync. On the other side, building a custom event platform from scratch is not realistic given that the same engineering team is running a parallel Kubernetes migration with limited headcount.

The hybrid uses a managed event broker, Kafka on Confluent Cloud or AWS MSK, as the integration backbone. A lightweight iPaaS layer, either MuleSoft or AWS EventBridge, sits alongside it to handle protocol translation and connector management for the legacy systems that cannot publish events natively. This gives BrightKids dead-letter queues, schema registry, replay capability, and standardised retry logic without the overhead of building and operating that infrastructure from scratch.

Migration is sequential. Integrations are triaged by failure impact first. The warehouse inventory sync and product catalogue update flows caused the most severe incidents and move to the event platform first. During transition, legacy connectors run in parallel with their event-driven replacements until production parity is confirmed, at which point the old connector is decommissioned. Lower-risk integrations follow in waves across the 18-month window. The 120+ point-to-point integrations do not all migrate at once. That would be the rip-and-replace approach the business cannot afford.

The platform's schema registry enforces contracts between producers and consumers, which directly addresses the pricing mismatch problem. A product catalogue update in one region cannot propagate inconsistent data downstream if the schema contract is validated at publish time.

Q4. KPIs

The following metrics track modernisation progress against the baselines and SLA targets stated in the case.

Mean Time to Recovery sits at 16 hours based on the US order delay. The target is under 1 hour. Getting there requires two things working together: centralised observability that detects failures within minutes rather than hours, and GitOps-driven rollback that makes resolution a pipeline trigger rather than a manual investigation and redeploy cycle.

Order API Uptime is currently around 98.5% against a retailer SLA of 99.9%. That gap is roughly 65 hours of additional downtime per year, significant enough that the business is likely already paying for it in SLA penalties. Multi-region Kubernetes with HPA, readiness probes routing traffic away from unhealthy pods, and blue/green deployments eliminating deployment-caused outages are what close that gap.

Patching Cycle is 11 days globally for a critical CVE. The target is under 24 hours. With the centralised image pipeline, this stops being a coordination problem and becomes a pipeline outcome. One base image update triggers automated rebuilds globally.

Deployment Frequency moves from manual and infrequent to weekly or on-demand with blue/green or canary patterns. The reason deployments are currently infrequent is not discipline, it is risk. Manual deployments fail unpredictably. GitOps with automated rollback removes that risk, and canary deployments allow partial traffic exposure before full rollout so on-demand releases become safe by default.

Integration Success Rate is not currently tracked because the point-to-point integrations produce no observability. Failures are only discovered when downstream systems break. The target is 99.5% successful event delivery with 100% of failures captured in dead-letter queues with a full audit trail. The introduction of this metric is itself a modernisation outcome: it is only measurable once the event platform is in place.

Security Posture is tracked as a set of binary checkpoints rather than trend metrics: zero externally reachable admin endpoints, 100% of deployments through the signed image pipeline, and all penetration test findings resolved within defined SLA. These confirm the governance framework is structurally enforced. If any of these fail, the framework has a gap that needs closing.